



Time to read: 8–10 minutes

Topic: *AI SDLC phases and what happens in each*

Big idea: AI SDLC is a **loop**, not a straight line.

Why AI SDLC is different

Traditional SDLC assumes code defines behaviour and deployments are relatively stable.

AI SDLC assumes **data and models define behaviour**, so systems must be built to handle:

- Changing data
- Uncertain outputs
- Continuous monitoring
- Safe iteration and retraining

That's why AI SDLC is best visualised as a cycle:

Problem → Data → Model → Deploy → Monitor → Iterate

AI SDLC Phases (What happens, what can go wrong and what “good” looks like)

1) Problem Definition and Requirement Analysis

What happens

This phase turns a vague idea into an engineering-ready scope.

You define:

- **Business objective** (what changes if we succeed?)
- **Decision point** (what decision will AI improve?)
- **Output type** (score, label, ranking, text)
- **Constraints** (latency, cost, compliance, explainability)

- **Success metrics** (business + model + system KPIs)

Common failure modes:

- Building AI where a rule would work better
- Misaligned metrics (optimise accuracy while business needs recall)
- No decision workflow (model output doesn't lead to action)

What good looks like:

A clear problem statement + measurable success criteria + integration plan.

2) Data Collection and Preprocessing

What happens

You identify and acquire data sources and turn raw data into a usable dataset.

You perform:

- Data sourcing (logs, sensors, transactions, documents)
- Cleaning (missing values, duplicates, outliers)
- Labelling strategy (if supervised)
- Train/val/test splitting
- Data quality checks and documentation

Common failure modes:

- Insufficient data or poor label quality
- Data leakage (future information accidentally used)
- Non-representative dataset → bias
- Inconsistent pre-processing between train and inference

What good looks like:

A versioned dataset with a data dictionary, quality checks and traceability.

3) Feature Engineering

What happens:

You translate raw signals into meaningful model inputs.

Examples:

- Rolling averages in time series
- User-level aggregates (30-day spend)
- Text vectorisation/embeddings
- Normalisation and encoding

Common failure modes:

- Features not available at inference time
- Overly complex features that break in production
- Leakage hidden in derived features

What good looks like:

Features that are:

- Stable
- Available in production
- Demonstrably useful

4) Model Development and Training

What happens:

You select an approach and train models using established frameworks.

Tasks include:

- Selecting baseline models first
- Training and tuning
- Experiment tracking
- Reproducibility controls (seeds, versions, configs)

Common failure modes:

- No baseline → no benchmark
- Overfitting to validation data
- Tuning without a disciplined plan
- “Notebook success” without production constraints

What good looks like:

A reproducible training pipeline and a model artefact ready for evaluation and packaging.

5) Model Evaluation and Validation

What happens

You validate the model against metrics and real-world expectations.

Evaluation includes:

- Model metrics (precision, recall, F1, calibration)
- Robustness testing (edge cases)
- Fairness/bias checks (where relevant)
- Latency/cost profiling (early)

Common failure modes:

- Using only accuracy
- Ignoring class imbalance
- No evaluation across segments (geography, devices, user cohorts)
- Not testing robustness against noisy inputs

What good looks like:

A release-ready evaluation report and a “model card” describing assumptions and risks.

6) Model Deployment and Integration

What happens:

The model becomes part of an application or service.

Deployment patterns:

- Batch scoring (nightly reports)
- Real-time API serving
- Edge deployment (on-device / on-prem)

Integration tasks:

- Containerisation
- Endpoint design (inputs/outputs)
- Security (auth, rate limiting)
- Compatibility (same pre-processing pipeline)

Common failure modes:

- Training/inference mismatch
- Latency too high for real-time needs
- No rollback mechanism
- Security and access control ignored

What good looks like:

A deployed, versioned model with stable inputs/outputs and a rollback strategy.

7) Monitoring and Maintenance

What happens:

You observe the system in the real world.

You monitor:

- System health: Latency, errors, throughput
- Data health: Drift, missingness, schema changes
- Model behaviour : Prediction distribution shifts

- Business impact: Adoption, time saved, losses prevented

Common failure modes:

- No monitoring → silent failure
- Waiting for labels before noticing issues
- Not collecting feedback signals
- No alerts or response playbooks

What good looks like:

Dashboards + alerts + defined actions when thresholds are crossed.

8) Ethical and Security Compliance

What happens:

You ensure the AI system is safe, fair and compliant.

This includes:

- Privacy handling (data minimisation, retention)
- Bias/fairness evaluation
- Explainability needs (where required)
- Model security risks (prompt injection in Gen AI, adversarial input)
- Auditability and documentation

Common failure modes:

- Treating ethics as a checklist at the end
- No audit trail for decisions
- Unclear accountability for failures

What good looks like:

Compliance is integrated into the lifecycle, not appended later.

9) Model Iteration (Continuous Improvement Loop)

What happens:

You decide whether and how to update the model.

Iteration triggers:

- Drift detected
- Performance degradation
- New data available
- Business objective changes
- Cost/latency optimisation needs

Iteration actions:

- Data refresh
- Feature updates
- Retraining or fine-tuning
- Safe deployment (blue-green/canary)
- Monitor again

Common failure modes:

- Retraining without diagnosing the root cause
- Changing too many things at once
- No version control → can't reproduce/rollback

What good looks like:

Controlled iteration with clear comparisons, approvals and rollback readiness.

Summary: The AI SDLC in one line

AI SDLC is a continuous loop in which **data and models** change over time, so the system must be engineered for monitoring, iteration and control.

Quick Exercise (2 minutes)

Pick one AI use case (fraud, recommendations, predictive maintenance) and fill this mini-map:

1. Problem definition: What decision improves?
2. Data sources: What signals exist?
3. Output: Score/label/ranking/text?
4. Deployment: Batch/API/edge?
5. Monitoring: What would you alert on?